

Pandas lisible

9 réécritures chaînées



.pipe, .assign, reshape : le code chaîné qu'on relit, avant/après et pourquoi.

Un df.py qu'on relit en revue de code ne devrait jamais forcer le lecteur à remonter cinq lignes pour savoir ce que contient df3. La chaîne pandas n'est pas du style : c'est un contrat de lecture, chaque étape se lit dans l'ordre où elle s'exécute. Ce guide te donne 9 réécritures avant/après, avec le pourquoi qui tient en revue de code.

Colonnes et fonctions : construire sans casser.

Ici, une colonne bien créée se lit en une fois, sans réaffectation qui oblige à remonter le fil.

1. .assign() pour créer des colonnes sans réaffecter df

Cas : Un data analyst calcule une marge et un pourcentage de marge sur un export de ventes, avant de ne garder que les lignes rentables.

```
# avant, illisible
df['margin'] = df['revenue'] - df['cost']
df['margin_pct'] = df['margin'] / df['revenue']
df = df[df['margin_pct'] > 0]
```

```
# apres, chaine
df = (
    df
    .assign(
        margin=lambda d: d['revenue'] - d['cost'],
        margin_pct=lambda d: d['margin'] / d['revenue'],
    )
    .query('margin_pct > 0')
)
```

Correction : assign renvoie une copie à chaque appel, sans réaffectation ligne par ligne qui mute df en place. Le lambda d: référence la colonne margin créée deux lignes plus haut, dans le même bloc : le calcul reste groupé au lieu d'être éclaté sur trois lignes qu'un relecteur doit recoller mentalement.

2. .pipe() pour insérer une fonction maison dans la chaîne PIÈGE CLASSIQUE

Cas : Un pipeline de reporting mensuel nettoie les dates, calcule la cohorte d'un client et détecte le churn avant de sortir la liste des comptes à risque.

```
# avant, illisible
df2 = clean_dates(df)
df3 = add_cohort(df2)
df4 = flag_churn(df3)
result = df4[df4['is_churn']]
```

```
# apres, chaine
result = (
    df
    .pipe(clean_dates)
    .pipe(add_cohort)
    .pipe(flag_churn)
    .query('is_churn')
)
```

Correction : chaque `.pipe(fonction)` nomme une étape métier directement dans la chaîne. Verdict tranché : `.pipe()` bat `.apply()` ici, parce qu'un `.apply(lambda row: ...)` cache la logique dans une fonction anonyme illisible en revue de code, alors que `.pipe(add_cohort)` donne un nom à l'étape et permet de la tester isolément.

3. Chaînage vs variables intermédiaires df1/df2/df3

Cas : Une revue de code sur un script de reporting hérité, où trois variables numérotées masquent le fil réel du calcul.

```
# avant, illisible
df1 = df[df['country'] == 'FR']
df2 = df1.groupby('region').sum()
df3 = df2.reset_index()
df_final = df3.sort_values('revenue', ascending=False)
```

```
# apres, chaine
df_final = (
    df
    .query("country == 'FR'")
    .groupby('region').sum()
    .reset_index()
    .sort_values('revenue', ascending=False)
)
```

Décision : avec `df1`, `df2`, `df3`, un relecteur doit retenir quel état contient quelle variable et vérifier qu'aucune n'est réutilisée par erreur plus loin. La chaîne verrouille l'ordre de lecture et supprime les variables orphelines qui traînent en mémoire.

À retenir

Le nombre de `df` numérotés dans un fichier est un signal d'alerte en revue de code, pas un détail de style.

Observation terrain : sur un script de reporting hérité, un collègue a modifié `df2` en pensant corriger `df3`, parce que les deux étaient visuellement interchangeables. La chaîne aurait rendu cette confusion impossible.

Sélection et filtrage sûrs : la ligne qu'on modifie vraiment.

Ici, le bug le plus vicieux ne plante jamais : il modifie silencieusement la mauvaise copie.

4. `.loc[]` vs indexation chaînée : le `SettingWithCopyWarning` ÉLIMINATOIRE

Cas : Un filtre sur les clients français suivi d'un flag ajouté après coup, dans un notebook partagé entre trois personnes.

```
# avant, illisible et dangereux
df_fr = df[df['country'] == 'FR']
df_fr['flag'] = 1
```

```
# apres, chaine et sur
df_fr = df.loc[df['country'] == 'FR'].copy()
df_fr.loc[:, 'flag'] = 1
```

Correction : `df[...][...] = valeur` est une indexation chaînée. Pandas ne peut pas garantir si la deuxième opération modifie une vue sur `df` ou une copie : c'est le `SettingWithCopyWarning`, et le bug est silencieux, le code tourne sans erreur mais peut ne rien modifier du tout. `.loc[]` suivi d'un `.copy()` explicite lève l'ambiguïté.

5. `.query()` pour filtrer lisiblement

Cas : Un filtre à trois conditions sur pays, revenu et statut, dans une extraction ad hoc préparée pour un comité mensuel.

```
# avant, illisible
df[(df['country'] == 'FR') & (df['revenue'] > 1000) & (df['status'] != 'cancelled')]
```

```
# apres, chaine
df.query("country == 'FR' and revenue > 1000 and status != 'cancelled'")
```

Exemple : la version avec parenthèses et `&` répète `df[...]` trois fois et oblige à parenthéser chaque condition pour respecter la priorité des opérateurs Python. `.query()` se lit comme une phrase de filtre, sans bruit syntaxique, et accepte `@variable` pour injecter une valeur externe.

6. `.rename(columns=...)` au lieu de réaffectations

Cas : Un export CSV dont l'ordre des colonnes change d'un mois sur l'autre selon la version de l'outil source.

```
# avant, illisible et fragile
df.columns = ['id', 'date', 'rev', 'cost']
```

```
# apres, chaine
df = df.rename(columns={'revenue': 'rev', 'transaction_date': 'date'})
```

Correction : réaffecter `df.columns` avec une liste dépend de l'ordre exact des colonnes source. Si la source ajoute ou retire une colonne demain, l'affectation associe silencieusement les mauvais noms aux mauvaises colonnes. `.rename()` mappe explicitement l'ancien nom vers le nouveau : seule la colonne visée bouge.

3 réflexes anti-copie

Ce que tu dois garder en tête avant de committer une chaîne pandas.

Créer

`.assign()` sans muter en place.

Trancher

`.loc[]` `.copy()` entre vue et copie.

Renommer

`.rename(columns={})` sans dépendre de l'ordre.

Reshape et agrégation : une étape, pas cinq.

Ici, la lisibilité vient du fait qu'un calcul nommé dit exactement ce qu'il produit.

7. Reshape `.melt()/pivot_table()` en une étape

Cas : Un tableau de ventes par produit et par date à transformer en tableau croisé pour un dashboard hebdomadaire.

```
# avant, plusieurs etapes manuelles
wide = df.pivot(index='date', columns='product', values='sales')
wide = wide.fillna(0)
wide = wide.reset_index()
```

```
# apres, chaine avec agregation integree
summary = (
    df
    .pivot_table(index='date', columns='product', values='sales',
                 aggfunc='sum', fill_value=0)
    .reset_index()
)
```

Sortie : `.pivot()` échoue dès qu'il existe des doublons index/colonne et n'agrège rien, il faut alors dédupliquer à la main avant. `.pivot_table()` agrège avec `aggfunc` et gère les combinaisons manquantes avec `fill_value` dans le même appel.

8. `.groupby().agg()` avec named aggregation PIÈGE CLASSIQUE

Cas : Un récapitulatif par région avec chiffre d'affaires, nombre de clients et panier moyen pour une revue commerciale.

```
# avant, dict manuel qui se desynchronise
g = df.groupby('region')
result = pd.DataFrame({
    'total_revenue': g['revenue'].sum(),
    'nb_clients': g['client_id'].nunique(),
    'avg_basket': g['revenue'].mean(),
})
```

```
# apres, named aggregation
result = (
    df
    .groupby('region')
    .agg(
        total_revenue=('revenue', 'sum'),
        nb_clients=('client_id', 'nunique'),
        avg_basket=('revenue', 'mean'),
    )
    .reset_index()
)
```

Sortie : le piège classique est d'écrire `.agg(['sum', 'mean'])` sur plusieurs colonnes, ce qui crée des colonnes en MultiIndex illisibles au premier `print(df)`. La named aggregation fixe le nom de sortie et la paire colonne/fonction dans le même mot clé : le nom ne peut plus se désynchroniser du calcul.

9. `.astype('category')` pour la mémoire et la clarté

Cas : Un export de 2 millions de lignes où pays et statut sont des colonnes texte à faible cardinalité.

```
# avant, coute cher et rien ne l'indique
df['country'].dtype
# object, 2 000 000 lignes, une douzaine de valeurs distinctes
```

```
# apres, chaine
df = df.assign(
    country=lambda d: d['country'].astype('category'),
    status=lambda d: d['status'].astype('category'),
)
```

Correction : une colonne object à faible cardinalité stocke la chaîne complète à chaque ligne. category stocke les valeurs uniques une fois et remplace chaque ligne par un code entier. Observation terrain : sur un export de 2 millions de lignes, passer country et status en category a réduit l'empreinte mémoire nettement, et les groupby/sort_values sur ces colonnes sont devenus plus rapides.

9 sur 9. Verdict tranché à retenir : .pipe() documente une étape maison dans la chaîne, .apply(lambda row: ...) la cache. Un collègue qui relit ton code doit pouvoir suivre le fil de haut en bas sans ouvrir un débogueur : c'est le seul test qui compte avant de committer une chaîne pandas.

Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

